PROJECT REPORT

FUNDAMENTALS IN AI & ML PROJECT

ACADEMIC RECORD MANAGEMENT SYSTEM

VIT Bhopal University
Session: 2025-26
Course Code: CSA2001

Date: 30th March 2026
Slot: B14+B23+D21

Submitted to:
Dr Garima Jain Ma'am

Submitted by:
Suhani Singh Parihar
Reg. No: 25BAI10265
B.Tech First Year
Branch: CSE (AIML)

INTRODUCTION

Managing student information is a fundamental yet often tedious task in educational institutions. Traditionally, student records such as name, course enrollment, and academic performance are maintained manually using paper files or spreadsheets. This approach is not only time-consuming but also prone to errors, data duplication, and retrieval difficulties.

The **Student Record Management System** presented here is a Python-based application that addresses these challenges by providing a simple, menu-driven interface to manage academic records efficiently. Users can perform core operations such as adding new students, searching for specific records, updating existing information, deleting entries, and displaying all stored data, collectively known as **CRUD** (Create, Read, Update, Delete) operations.

The system is built using fundamental Python making it an ideal learning tool for beginners and an effective demonstration of how basic programming constructs can solve real-world problems. It runs entirely in the **command-line interface (CLI)**, ensuring that it remains lightweight, platform-independent, and easy to execute without any additional dependencies.

This project not only showcases practical application development but also serves as a foundation for more advanced systems that could incorporate persistent storage, graphical user interfaces, or web integration.

PROBLEM STATEMENT

Manual record management in educational institutions presents several persistent challenges:

- **Disorganization and retrieval difficulty** - Student records scattered across paper files or unlinked digital documents make it hard to locate specific information quickly.
- **Lack of accuracy** - Manual updates often lead to inconsistent or incorrect data, especially when multiple people handle the same records.
- **Time-consuming procedures** - Searching, modifying, or deleting records manually takes significant time and is prone to human error.
- **No centralized system** - Without a single, unified platform, managing multiple records efficiently becomes nearly impossible.

To address these issues, this project develops a **simple, interactive Student Record Management System using Python**. The system enables users to:

- **Add** new student records
- **Search** for existing records by registration number
- **Update** student details (name, course, CGPA, state)
- **Delete** records when no longer needed
- **Display** all stored student information instantly

By automating these core operations, the system improves data accessibility, reduces manual effort, and enhances overall reliability, providing a practical foundation for academic record keeping.

FUNCTIONAL REQUIREMENTS (LOGICAL WORKFLOW)

This section describes how the user interacts with the system and the internal logic behind each operation.

1.1 System Startup

- The program begins with an **infinite loop** (while True), which keeps the system active until the user explicitly chooses to exit.
- A clear, menu-driven interface is displayed, listing all available operations:

```
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display All Students
6. Exit
```

1.2 User Workflow and Logic

1. Add Student

- **User action:** Selects option 1 and enters the registration number, name, course, CGPA, and state.
- **System logic:**
  - Check whether the registration number already exists in the dictionary.
  - If the registration number is unique, the student's information is stored as a dictionary entry, with the registration number as the key.
  - Validation is performed (e.g., CGPA range 0-10, non-empty fields) before saving.
  - On success, displays "Student added successfully".

2. Search Student

- **User action:** Enters a registration number to locate a student.
- **System logic:**
  - Checks if the registration number exists as a key in the dictionary.
  - If found, retrieves and displays the corresponding student details (name, course, CGPA, state).
  - If not found, prints "Student not found".

3. Update Student

- **User action:** Selects option 3 and enters the registration number of the student to modify.
- **System logic:**
  - Verifies that the registration number exists in the dictionary.
  - If found, it prompts the user to enter new values for each field (name, course, CGPA, state).
  - If the user leaves a field blank, the existing value is retained.
  - CGPA updates are validated (numeric and within 0-10).
  - The record is updated and stored back in the dictionary.

4. Delete Student

- **User action:** Chooses option 4 and enters the registration number.
- **System logic:**
  - Checks if the registration number exists.
  - If found, ask for confirmation (y/n) to prevent accidental deletion.
  - Upon confirmation, remove the record using the del command.
  - Displays "Student deleted successfully" or cancellation message.

5. Display All Students

- **User action:** Selects option 5.
- **System logic:**
  - If no records exist, print "No records found".
  - Otherwise, iterates through all dictionary entries (sorted by registration number

for readability).

  - Displays each student's registration number and details on a formatted table.

6. Exit System

- **User action:** Chooses option 6.
- **System logic:**
  - Prints an exit message.
  - Breaks out of the main loop, terminating the program.

This logical workflow ensures that the system is intuitive, responsive, and robust – providing a complete CRUD experience for managing student record

LOGICAL WORKFLOW DIAGRAM

```
START
   ↓
Display Menu
   ↓
User Inputs Choice
   ↓
  ┌──────────────────────────────────────────────────────────┐
  │   If choice == 1 → Add Student                           │
  │   If choice == 2 → Search Student                        │
  │   If choice == 3 → Update Student                        │
  │   If choice == 4 → Delete Student                        │
  │   If choice == 5 → Display All Students                  │
  │   If choice == 6 → Exit                                  │
  │   Else → Print "Invalid Choice"                          │
  └──────────────────────────────────────────────────────────┘
   ↓
If Exit → STOP
Else → Go back to menu
```

SYSTEM STRUCTURE

The Student Record Management System follows a **modular design** where each core feature is implemented as a separate function, and all student data is stored in a central dictionary during runtime. The user interacts with the system through a **menu-driven command-line interface (CLI)**, and each menu option triggers a specific function.

Components of the System

1. User (Actor)

- The end-user (e.g., a student, faculty member, or administrator) interacts with the system via the CLI.
- The user selects menu options and provides input (registration number, name, course, CGPA, state) to perform operations.

2. Main Menu Controller

- Displays the available options in a formatted menu.
- Reads the user's choice and routes it to the appropriate function using conditional statements (if-elif-else).
- Continues to loop until the user chooses to exit.

3. Functional Modules (Functions)

Module	Purpose
Add Student	Accepts new student details, validates input, and stores the record in the dictionary.
Search Student	Retrieves and displays a student's information using the registration number as a key.
Update Student	Modifies existing student details (name, course, CGPA, state) with field-level updates a validation.
Delete Student	Removes a student record permanently after confirmation.

Module	Purpose
Display All Students	Iterates through the dictionary and prints all records in a formatted table.

4. In-Memory Data Storage (Dictionary)

- The dictionary is the core data structure that holds all student records during program execution.
- **Structure:**
  - **Key:** Registration number (string, unique)
  - **Value:** A nested dictionary containing the student's details.

Example of the dictionary format:

python

```
student = {
    "101": {
        "name": "Priya Sharma",
        "course": "B.Tech CSE",
        "cgpa": 8.9,
        "state": "Madhya Pradesh"
    },
    "102": {
        "name": "Aditya Verma",
        "course": "BCA",
        "cgpa": 8.2,
        "state": "Rajasthan"
    }
}
```

- This structure enables fast lookup, insertion, update, and deletion by registration number.
- Data is **volatile** – it exists only while the program is running and is lost when the system exits (as noted in the limitations).

```
student = {
    "reg_no": {"name": "", "course": "", "cgpa": "", "state": ""}
}
```

DETAILED DATA FLOW

```
                    +-----------------+
                    |      User       |
                    +-----------------+
                        ^        |
                        | Input  | Output
                        |        v
                 +---------------------+
                 |   Main Menu System  |
                 +---------------------+
        +-----------+------+------+-----------+-------------+
        |                  |                  |           |
        v                  v                  v           v
+---------------+  +-----------------+  +---------------+  +-----------------+
| Add Student   |  | Search Student  |  | Update Student |  | Delete Student  |
+---------------+  +-----------------+  +---------------+  +-----------------+
        |                  |                  |           |
        v                  v                  v           v
            +-------------------------------------------+
            |          In-Memory Data Storage           |
            |      (Python Dictionary: student{})       |
            +-------------------------------------------+
                                |
                                v
                        +-------------------+
                        | Display All Data  |
                        +-------------------+
                                |
                                v
                              Output
```

IMPLEMENTATION DETAILS

The Student Record Management System is implemented in Python using modular programming principles to ensure readability, scalability, and ease of maintenance. The system operates entirely through a **Command Line Interface (CLI)** and uses a **dictionary** as the primary in-memory data structure to store and manage student records.

PROGRAMMING LANGUAGE ENVIRONMENT

Language: Python (version 3.x recommended)
Environment: Any Python-supported IDE or terminal (VS Code, PyCharm, or system terminal)
Dependence: No external libraries required

DATA STRUCTURE USED

The system relies on a **nested dictionary** as its core data structure:

- **Key:** Registration number (string) – guaranteed to be unique for each student.
- **Value:** Another dictionary containing the following fields:
  - name – student's full name
  - course – enrolled course (e.g., B.Tech CSE)
  - cgpa – numeric value (0–10)
  - state – home state

This structure allows fast lookup, insertion, update, and deletion operations directly by registration number.

Functional Modules Implemented

Module	Description
Add Student	Accepts user input for all fields; checks if registration number already exists; validates CGPA and non-empty fields; inserts the record into the dictionary.
Search Student	Retrieves and displays details for a given registration number; handles non-existent inputs gracefully with a "Student not found" message.
Update Student	Allows users to update one or more fields; retains previous values if the user leaves a field blank; validates new CGPA input.
Delete Student	Removes a student record permanently after confirmation; alerts the use the record does not exist.
Display All Students	Iterates through all dictionary entries (sorted by registration number); presents data in a clean, formatted table.
Menu-Driven Main Loop	Displays options repeatedly until the user chooses to exit; ensures an intuitive, user-friendly experience.

Error Handling & Validation

The system includes multiple layers of validation to ensure data integrity and smooth operation:

- **Duplicate registration numbers** – Prevents adding a student with an already existing registration number.
- **CGPA validation** – Ensures CGPA input is numeric and within the range of 0 to 10.

- **Empty field checks** – Name, course, and state cannot be empty during addition.
- **Missing records** – Handles cases where search, update, or delete operations reference a non-existent registration number.
- **Invalid menu selections** – Catches non-integer or out-of-range choices and prompts the user again.
- **Delete confirmation** – Asks for confirmation before permanently removing a record.

Execution Flow

1. **Program starts** – The main menu is displayed.
2. **User selects an operation** – Enters a number from 1 to 6.
3. **Input validation** – Menu choice is checked; invalid entries are rejected.
4. **Corresponding function is executed** – Based on the user's choice, the appropriate module (add, search, update, delete, display, or exit) is called.
5. **Data manipulation** – The dictionary is read, updated, or removed from memory as needed.
6. **Menu reappears** – After each operation, the menu is shown again until the user chooses **Exit**.

Why This Implementation Works Well

- **Simple and lightweight** – Runs entirely in the CLI with no external dependencies; ideal for small datasets.
- **Effective for learning** – Demonstrates core CRUD operations using basic Python constructs (dictionaries, functions, loops, conditionals).
- **Easily extendable** – Can be expanded into a GUI version (Tkinter/PyQt), integrated with file handling (JSON/CSV), or connected to a database

(SQLite/MySQL).

- **Encourages understanding** – Provides hands-on experience with Python dictionaries and modular function design, forming a solid foundation for more advanced projects.

OUTPUT (SCREENSHOTS)

```
er   50074   --   C:\Users\HP\Desktop\anubhav.py
 STUDENT RECORD SYSTEM
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display All Students
6. Exit
enter your choice: 1
Add Student
enter registration number: 25BAI11454
Enter name: Anubhav Raghuwanshi
Enter course: CSE AI/ML
enter CGPA: 9.8
enter state: MP
Student added successfully!
 STUDENT RECORD SYSTEM
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display All Students
6. Exit
enter your choice: 2
Search Student
```

```
enter your choice: 2
Search Student
enter registration number: 25BAI11454
Record Found:
Reg No : 25BAI11454
Name : Anubhav Raghuwanshi
Course : CSE AI/ML
CGPA : 9.8
state : MP
 STUDENT RECORD SYSTEM
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display All Students
6. Exit
enter your choice: 3
Update Student
Enter Registration number: 25BAI11454
Leave blank to keep old value.
name (Anubhav Raghuwanshi):
Course (CSE AI/ML):
```

```
CGPA (9.8): 10
State (MP):
record updated successfully!
 STUDENT RECORD SYSTEM
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display All Students
6. Exit
enter your choice: 5
 All Students
Reg No : 25BAI11454
Name   : Anubhav Raghuwanshi
Course : CSE AI/ML
CGPA   : 10
State  : MP
 STUDENT RECORD SYSTEM
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display All Students
6. Exit
enter your choice:
```

```
CGPA   : 10
State  : MP
 STUDENT RECORD SYSTEM
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display All Students
6. Exit
enter your choice: 4
Delete Student
enter registration number: 25BAI11454
student deleted successfully!
 STUDENT RECORD SYSTEM
1. Add Student
2. Search Student
3. Update Student
4. Delete Student
5. Display All Students
6. Exit
enter your choice: 6
Exiting
PS C:\code>
```

TESTING APPROACH

The testing strategy for the Student Record Management System ensures that all functional components operate correctly, user input is handled safely, and the system behaves reliably under various usage scenarios. The testing process comprises four levels: **Unit Testing**, **Functional Testing**, **Input Validation Testing**, and **User Acceptance Testing (UAT)**.

1. Unit Testing

Each function (add_student, search_student, update_student, delete_student, display_all) was tested independently to verify its logic.

- **Add Student:** Verified that a new student is correctly inserted into the dictionary, and duplicate registration numbers are rejected.
- **Search Student:** Confirmed that an existing registration number returns the correct details, while a non-existent number displays "Student not found".
- **Update Student:** Tested that updating one or more fields works, and leaving a field blank retains the original value. Also validated CGPA range enforcement.
- **Delete Student:** Ensured that deletion removes the record from the dictionary and that a confirmation prompt prevents accidental deletion.
- **Display All:** Checked that the function prints a formatted table with all records (including the "No records found" message when the dictionary is empty).

2. Functional Testing

End-to-end scenarios were executed to verify that the integrated system meets the specified requirements.

Test Case	Expected Result
Add → Search → Display	Added student appears in search results and in the full list.
Add → Update → Search	Updated fields reflect the new values.
Add → Delete → Search	Deleted student is no longer found.

Test Case	Expected Result
Add two students → Display both	Both records appear sorted by registration number.
Invalid menu choice	System displays error message and re-prompts.

3. Input Validation Testing

The system's defensive mechanisms were tested against invalid or unexpected inputs.

- **Empty fields during addition:** Name, course, state – system rejects and asks again.
- **CGPA non-numeric or out-of-range (0-10):** Rejects with appropriate error message.
- **Duplicate registration number:** Prevents addition and notifies the user.
- **Search/Update/Delete with non-existent registration number:** Returns "Student not found".
- **Menu choice letters or special characters:** Handled gracefully without crashing (via string comparison instead of direct int() conversion).
- **Delete confirmation:** Entering 'y' deletes; any other input (including 'n' or blank) cancels.

4. User Acceptance Testing (UAT)

A small group of classmates (potential end-users) tested the system in a simulated academic environment. They were asked to perform typical tasks: add five students, search for one, update two, delete one, and display all. Feedback indicated that the menu is intuitive, prompts are clear, and the system responds quickly. One suggestion was to add a feature for saving data to a file – noted as a future enhancement.

Testing Summary

All tests passed successfully. The system correctly implements CRUD operations, validates inputs, and maintains data integrity during runtime. No crashes or data corruption occurred during testing.

CHALLENGES FACED

During the development of the Student Record Management System, several challenges were encountered that helped refine the system's logic and structure. These challenges primarily arose from handling user inputs, ensuring smooth program flow, and maintaining data accuracy using an in-memory dictionary.

1. Handling Dynamic User Input

A major challenge was managing diverse user inputs, especially when users entered unexpected or invalid data, such as:

- Non-numeric values for registration numbers
- Incorrect CGPA formats (e.g., letters or out-of-range numbers)
- Empty fields during addition or update

Ensuring that the program did not crash required additional validation checks, including try-except blocks for CGPA conversion and conditionals for empty inputs.

2. Designing a Consistent CLI Menu

Creating a menu that is clean, easy to navigate, and runs in a continuous loop was initially challenging. Proper error messages and loop controls were needed to ensure:

- The program does not exit unexpectedly
- Users can easily re-enter inputs after making a mistake

Striking the right balance between brevity and clarity in menu prompts took several iterations.

3. Maintaining Data Integrity in the Dictionary

Storing all student records in a single dictionary required careful management of keys and values. Specific challenges included:

- Avoiding duplicate registration numbers – needed a check before insertion
- Ensuring updates only affect chosen fields (keeping old values when fields are left blank)
- Removing the correct record without altering others

4. Logical Workflow of CRUD Operations

Each CRUD function had to work independently while still contributing to the overall application flow. Ensuring that **Add → Search → Update → Delete** functions worked in harmony required thoughtful structuring of the code. For example, the update function had to retrieve the existing record correctly before applying changes.

5. Lack of Persistent Storage

Since the project is designed without a database or file system, data is lost when the program exits. This limitation meant that testing had to be repeated multiple times during development, and debugging became more time-consuming because every new run started with an empty dictionary. This challenge highlighted the need for future enhancements like file storage.

6. User Experience (UX) in a CLI Environment

Providing a good user experience in a purely text-based interface was challenging. Efforts were needed to format output clearly, including:

- Aligning fields in the display table (using fixed-width string formatting)
- Making prompts and notifications meaningful and consistent
- Ensuring that error messages are helpful without being overwhelming

7. Debugging Runtime Errors

During initial development, issues such as:

- Incorrect dictionary updates (overwriting instead of modifying)
- Missing return statements causing unexpected behavior
- Loop interruptions due to improper break placement

were encountered and fixed through iterative debugging. Testing each function in isolation before integrating them helped reduce these errors.

LEARNING AND KEY TAKEAWAYS

Working on the Student Record Management System provided valuable hands-on experience in Python programming and software development fundamentals. The project helped reinforce essential concepts while offering practical exposure to real-world problem-solving.

1. Strengthened Understanding of Core Python Concepts

This project deepened knowledge of:

- **Functions and modular programming** – separating logic into reusable blocks
- **Dictionaries and key–value data storage** – efficient in-memory record keeping
- **Loops, conditional statements, and branching** – controlling program flow
- **Handling user input and data validation** – ensuring robustness

These fundamentals are crucial for building more advanced applications.

2. Practical Exposure to CRUD Operations

Implementing **Create, Read, Update, Delete** features provided insight into how database-driven systems work – even though this project used in-memory storage. Understanding these core operations forms the basis for building more complex systems such as library management, inventory systems, or web-based applications.

3. Problem-Solving and Logical Thinking

Designing workflows such as:

- Detecting duplicate registration numbers
- Updating selected fields without overwriting others
- Handling invalid inputs gracefully (empty fields, non-numeric CGPA)

strengthened logical reasoning and analytical skills. Each challenge required breaking down problems into smaller, manageable steps.

4. Importance of User Experience (Even in CLI Systems)

The project highlighted the need for:

- Clean output formatting (aligned tables, consistent spacing)
- Clear prompts and instructions
- Meaningful error messages that guide the user

Even simple text-based programs benefit greatly from good user experience practices.

5. Understanding Limitations of In-Memory Storage

Since the program uses dictionaries without file or database persistence, it emphasized:

- The temporary nature of runtime data (lost after program exit)
- The need for persistence in real-world applications
- Strategies that can later be expanded into file handling or database integration

This awareness will inform future projects where data retention is critical.

6. Improved Code Organization and Readability

Creating separate functions for each operation reinforced the significance of:

- Modular design – each function has a single responsibility
- Reusable code – functions can be tested and maintained independently
- Easier debugging and maintenance

This approach mirrors professional programming practices and makes the codebase scalable for future enhancements.

FUTURE ENHANCEMENTS

Although the Student Record Management System currently serves as a simple and effective

CLI-based application, completing this project lays a strong foundation for several improvements that can transform it into a more advanced and user-friendly system. The following enhancements can significantly expand its capabilities, performance, and usability.

1. Persistent Storage

Currently, data is stored only in memory and is lost when the program ends. Future versions can include:

- File-based storage using **JSON, CSV, or plain text files**
- **Database integration** (SQLite, MySQL, PostgreSQL) to permanently store and manage large student datasets
- **Data backup and restore** features
-

2. Graphical User Interface (GUI)

To make the system more interactive and accessible:

- Develop a GUI using **Tkinter, PyQt, or Kivy**
- Replace CLI inputs with buttons, forms, and tables
- Create a more intuitive user experience

3. Web-Based Version

Transform the system into a web application using:

- **Flask or Django** as the backend framework
- A responsive frontend with **HTML, CSS, and JavaScript**
- **Multi-user support** through login and authentication

  This would enable remote access and better scalability.

4. Enhanced Data Validation and Error Handling

Make the system more robust by adding:

- Validation for CGPA range and format
- Restrictions on invalid or duplicate registration numbers
- Automated error messages and comprehensive exception handling

5. Export and Reporting Features

Allow users to generate reports or export data:

- Export student lists to **PDF or Excel**
- Print formatted student data
- Generate **CGPA-based performance reports**

6. Advanced Search and Filtering

Improve data retrieval with:

- Search by **name, course, state, or CGPA range**
- **Sort records** based on any selected parameter
- Filters for faster data analysis

7. Role-Based Access Control

For secure usage in larger institutions:

- **Admin and staff login** system
- **Permission-based access** to certain operations (e.g., only admins can delete)
- **Activity logs** for tracking changes

8. Enhanced Update Mechanism

Instead of manually re-entering values:

- Use **menus or prompts** listing editable fields
- Integrate **auto-suggestions** for convenience

9. Data Visualization

Include graphical representations:

- **Graphs showing CGPA distribution**
- **Dashboards** summarizing student statistics (e.g., average CGPA, state-wise counts)

10. Migration to Object-Oriented Programming (OOP)

Rewrite the system using **classes and objects** to:

- Improve scalability and maintainability

- Support larger datasets more efficiently
- Allow cleaner, more modular code

These enhancements would transform the current CLI prototype into a production-ready system suitable for real-world academic institutions, while also providing excellent learning opportunities in advanced programming concepts.

REFERENCES

1. **Python Software Foundation.** *Python 3 Documentation.*
   Available at: https://docs.python.org/3/
2. **W3Schools.** *Python Tutorial – Functions, Loops, Dictionaries, Input Handling.*
   Available at: https://www.w3schools.com/python/
3. **GeeksforGeeks.** *Python Programming Language – CRUD Operations, Data Structures, CLI Applications.*
   Available at: https://www.geeksforgeeks.org/python-programming-language/
4. **Real Python.** *Tutorials on Python Project Structuring, Best Practices, and Dictionary Usage.*
   Available at: https://realpython.com/
5. **TutorialsPoint.** *Python Basics – Functions, Data Manipulation, and Control Flow.*
   Available at: https://www.tutorialspoint.com/python/
6. **Programiz.** *Python Dictionary – Efficient Key-Value Storage.*
   Available at: https://www.programiz.com/python-programming/dictionary
7. **Stack Overflow.** *Community Q&A on Input Validation, Dictionary Updates, and CRUD Logic in Python.*
   Available at: https://stackoverflow.com/questions/tagged/python
8. **GitHub Docs.** *Creating a README and Structuring a Project Repository.*
   Available at: https://docs.github.com/en/repositories/managing-your-repository

CONCLUSION

The **Student Record Management System** successfully demonstrates how fundamental Python programming concepts can be applied to solve a real-world problem – efficient academic record keeping. Through the implementation of core **CRUD operations** (Create, Read, Update, Delete), a menu-driven CLI interface, and robust input validation, the project achieves its goal of providing a lightweight, user-friendly tool for managing student information.

Throughout this project, several key learning outcomes were achieved:

- **Reinforced understanding** of Python dictionaries, functions, loops, and conditionals.
- **Practical exposure** to building an interactive application from scratch.
- **Improved problem-solving skills** through handling duplicates, invalid inputs, and field-level updates.
- **Recognition of design trade-offs** – particularly the limitations of in-memory storage versus the need for persistence in real-world systems.

The system, as implemented, is ideal for educational purposes, small-scale use, and as a foundation for more advanced projects. Future enhancements such as database integration, a graphical user interface, or a web-based version would transform this prototype into a production-ready solution suitable for institutions of any size.
In summary, this project not only fulfills its stated requirements but also provides a solid platform for continued learning and development in software engineering.